

Subject: Client Side Scripting	Subject Code: 316005
Semester: 06	Course: CO6K-A
Laboratory No: L001A	Name of Subject Teacher: Prof. Sneha Patange
Name of Student: Vedant Atul Shirke	Roll ID: 23203A0018

Experiment No:	14
Title of Experiment:	Write a program to handle data using React form.

I. Practical Significance:

A Simple React program that demonstrates how to handle form data — including reading input, updating state, and displaying the submitted data.

➤ Code

- Write a React program to design a simple form using React components that accepts user details such as name, email and password. Also displays them after form submission.

Code:

App.jsx:

```
import React, { useState } from "react";
import ReactDOM from "react-dom/client";
function RegistrationForm() {
  const [userData, setUserData] = useState({
    name: "",
    email: "",
    password: ""
  });
  const [submittedData, setSubmittedData] = useState(null);
  const handleSubmit = (event) => {
    event.preventDefault();
    console.log(userData);
    alert("Form Submitted Successfully");
    // Store submitted data
```

```
setSubmittedData(userData);  
// Clear the form  
setUserData({  
  name: "",  
  email: "",  
  password: ""  
});  
};  
return (  
  <div style={{ textAlign: "center", marginTop: "40px" }}>  
    <h2>Registration Form</h2>  
    <form onSubmit={handleSubmit}>  
      <div>  
        <label>Name: </label>  
        <input  
          type="text"  
          value={userData.name}  
          onChange={(e) =>  
            setUserData({ ...userData, name: e.target.value })  
          }  
        />  
      </div>  
      <br />  
      <div>  
        <label>Email: </label>  
        <input  
          type="email"  
          value={userData.email}  
          onChange={(e) =>  
            setUserData({ ...userData, email: e.target.value })  
          }  
        />  
      </div>  
      <br />  
      <div>  
        <label>Password: </label>  
        <input  
          type="password"  
          value={userData.password}  
          onChange={(e) =>  
            setUserData({ ...userData, password: e.target.value })  
          }  
        />  
      </div>  
      <br />  
      <button type="submit">Submit</button>  
    </form>  
    <br />  
    {submittedData && (  

```

```
    <div>
      <h3>Submitted Data</h3>
      <p>Name: {submittedData.name}</p>
      <p>Email: {submittedData.email}</p>
      <p>Password: {submittedData.password}</p>
    </div>
  )}
</div>
);
}
```

export default RegistrationForm;

index.jsx:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import FormExample from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <FormExample />
  </React.StrictMode>,
)
```

Output:

Registration Form

Name: Email: Password:

Registration Form

Name:

Email:

Password:

Submitted Data

Name: Vedant
Email: vedantashirke18@gmail.com
Password: 123456

II. Practical Related Questions:

1. Create a registration form that collects name, email, and password, and clears inputs after submission.

Ans:

Code:-

```
import React, { useState } from "react";
import ReactDOM from "react-dom/client";
function RegistrationForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const handleSubmit = (event) => {
    event.preventDefault();
    alert("Registration Successful!");
    console.log("Name:", name);
    console.log("Email:", email);
    console.log("Password:", password);
    // Clear the input fields
    setName("");
    setEmail("");
    setPassword("");
  };
  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
      <h2>Registration Form</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Name: </label>
          <input
            type="text"
```

App.jsx:

```
value={name}
      onChange={(e) => setName(e.target.value)}
      required
    />
  </div>
  <br />
  <div>
    <label>Email: </label>
    <input
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      required
    />
  </div>
  <br />
  <div>
    <label>Password: </label>
    <input
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      required
    />
  </div>
  <br />
  <button type="submit">Register</button>
</form>
</div>
);
}
export default RegistrationForm;
```

index.jsx:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import RegistrationForm from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <RegistrationForm />
  </React.StrictMode>,
)
```

Registration Form

Name: Email: Password:

Name: shreyas	index-B79Re5da.js:32
Email: shreyas@gmail.com	index-B79Re5da.js:32
Password: 12345	index-B79Re5da.js:32

Registration Form

Name: Email: Password:

2. Demonstrate how to handle form data dynamically using an object to store multiple input fields.

Ans:-

To handle form data dynamically using an object, we store all input values inside a single state object and update the object whenever any input field changes. This is very common in React forms and helps manage multiple fields easily.

Output:-

Form data can be handled dynamically by storing multiple input fields inside a single state object. Each input field uses the name attribute, and a common handleChange function updates the corresponding value using [name]: value. The spread operator (...) keeps previous values while updating the modified field. This approach simplifies form management in React.

Code:-**App.jsx:**

```
import React, { useState } from "react";
import ReactDOM from "react-dom/client";

function RegistrationForm() {

  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: ""
  });

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value
    });
  };

  const handleSubmit = (event) => {
    event.preventDefault();

    alert("Registration Successful!");

    console.log("Name:", formData.name);
    console.log("Email:", formData.email);
    console.log("Password:", formData.password);

    // Clear the form
    setFormData({
      name: "",
      email: "",
      password: ""
    });
  };

  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
      <h2>Registration Form</h2>
    </div>
  );
}
```



```
<form onSubmit={handleSubmit}>

  <div>
    <label>Name: </label>
    <input
      type="text"
      name="name"
      value={formData.name}
      onChange={handleChange}
      required
    />
  </div>

  <br />

  <div>
    <label>Email: </label>
    <input
      type="email"
      name="email"
      value={formData.email}
      onChange={handleChange}
      required
    />
  </div>

  <br />

  <div>
    <label>Password: </label>
    <input
      type="password"
      name="password"
      value={formData.password}
      onChange={handleChange}
      required
    />
  </div>

  <br />

  <button type="submit">Register</button>

</form>
</div>
);
}

export default RegistrationForm;
```

index.jsx:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import RegistrationForm from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <RegistrationForm />
  </React.StrictMode>,
)
```

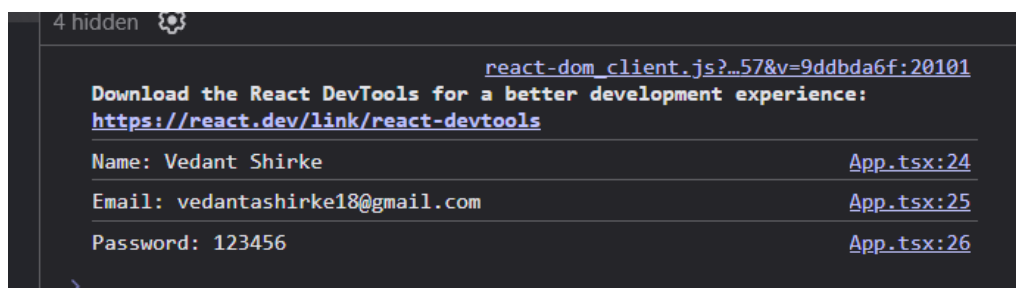
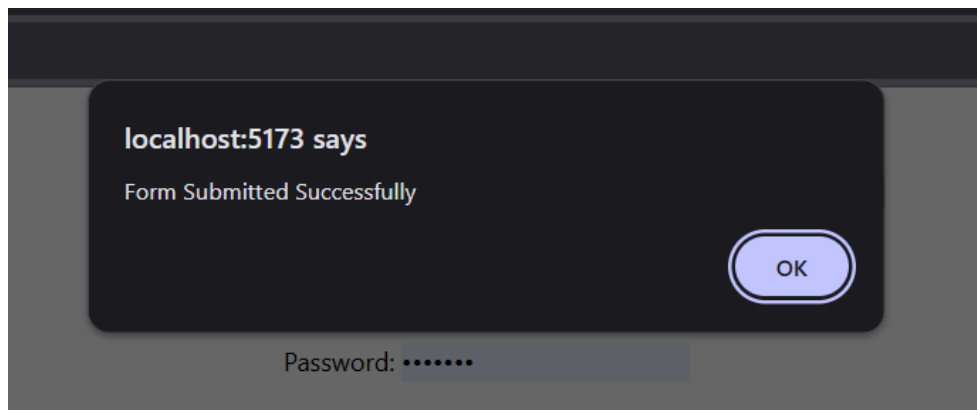
Output:-

Registration Form

Name:

Email:

Password:



Registration Form

Name:

Email:

Password:

```
4 hidden 
react-dom_client.js?...57&v=9ddbda6f:20101
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
Name: Vedant Shirke App.tsx:24
Email: vedantashirke18@gmail.com App.tsx:25
Password: 123456 App.tsx:26
>
```

III. Exercise:

1. How do you handle form data using React Hooks like `useState` and `useEffect`.

Ans:

In React, form data can be handled using Hooks such as `useState` and `useEffect`. The `useState` hook is used to store and manage the values entered in form input fields like name, email, and password. When a user types in an input field, the state is updated dynamically, allowing React to keep track of the current form data.

The `useEffect` hook is used to perform side effects whenever the form data changes. For example, it can be used to validate input fields, display messages, or log changes to the console. By monitoring the state variables, `useEffect` runs automatically whenever the form data is updated.

Using these hooks together helps in efficiently managing form inputs, updating the UI dynamically, and performing actions based on user input in React applications..

2. Demonstrate how to handle form data dynamically using an object to store multiple input fields.

Ans:

In React, form data can be handled dynamically by storing multiple input field values inside a single object in the component's state. Instead of creating separate state variables for each field (such as name, email, and password), all values are stored as properties of one object.

Each input field is given a name attribute, and a common change handler function is used

to update the object whenever the user types in any field. The handler identifies which

field changed using the name attribute and updates only that specific property in the object while keeping the other values unchanged.

This approach makes form handling more efficient, scalable, and easier to manage, especially when dealing with large forms containing many input fields. It reduces repetitive code and allows a single function to manage updates for multiple inputs dynamically.

IV. Conclusion:

This practical demonstrates how React forms can be used to collect user input, manage form data using state, and process the submitted information. It helps in understanding how React handles dynamic data updates and improves interaction between the user interface and application logic.

Marks Obtained			Dated Signature of Teacher
Process Related (15)	Product Related (10)	Total (25)	